

LedgerLens

AI Systems Architecture Review

Reconciles cheaply first, reasons expensively last, and explains every decision



Executive summary

What it is

- A reconciliation engine for messy bank, ledger and processor files: exact, rule and fuzzy tiers resolve the obvious pairs; a masked, cost-capped LLM adjudicates only the ambiguous middle band.
- A LangGraph review gate pauses the run at a real interrupt() until a named reviewer resolves every open task, resumable from any process.

What it proves

- 105 tests, 6 / 6 golden checks, a review gate resumed 1 / 1 times cross-instance, 0 raw counterparty or reference strings leaked to the model or the MCP server (asserted against real sample data).
- Atomic runs with rollback; a Go match-worker validated by replaying real Python-exported events.

Why it is enterprise-ready

- Frozen interface contracts per design doc (docs/05), a single offline gate (`make check``: unit + golden + card-drift), and a numbered decision log for every consequential trade-off.
- Masking is one module used by both the LLM request and the human reviewer's tools — one perimeter, two consumers.

Ask of the audience: agree the pilot scope — which live LLM backend to validate first (Ollama, vLLM, Bedrock or Anthropic), the Postgres migration path, and whether the Kafka streaming profile is wanted for this client.

The problem and the users

The reconciliation problem

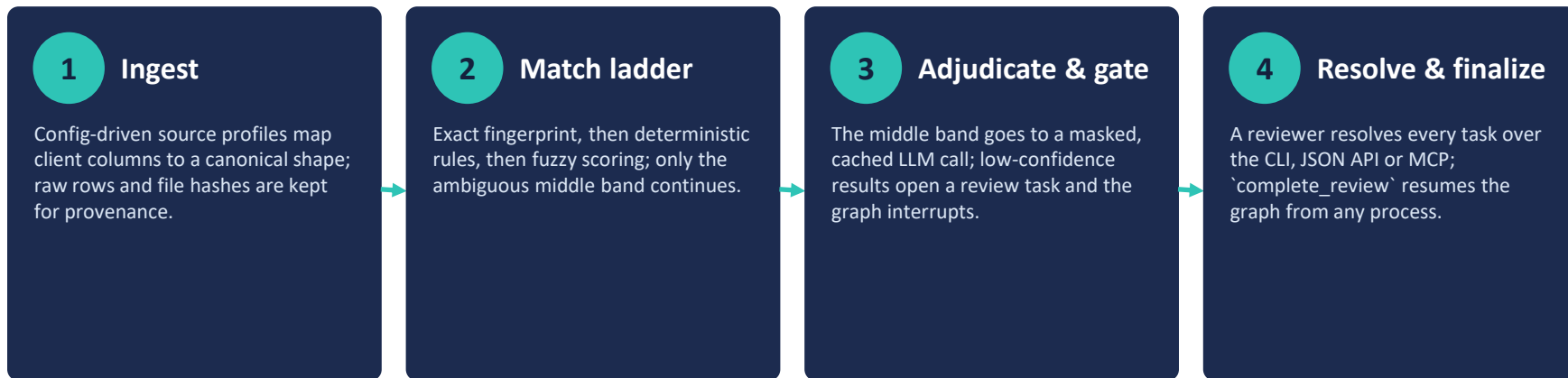
Bank statements, ledger exports and processor files never share a schema, a sign convention or a posting date. The obvious matches are cheap to automate; the cost is the residue — pairs that look alike but are not, and pairs that are the same transaction wearing two descriptions. Sending every pair to an LLM is neither affordable nor auditable.

The constraints

A decision that reaches an auditor must be explainable and reproducible. Raw counterparty and reference strings must not leave the perimeter unmasked. A low-confidence decision must stay open until a named human resolves it — the run cannot silently mark itself complete.

Actor	Needs	Frequency
Reconciliation analyst	Reviews exceptions, resolves review tasks with a decision and notes, completes the run	Per run
Controller / compliance	Match rate, review-required rate, an audit trail from decision back to source rows	On review
Implementation engineer	Onboards a new client source via a mapping profile, not a code change	Per client
Automation / AI assistant	The same review gate over MCP (7 tools) or the JSON API, never raw source rows	Ad hoc

Solution at a glance



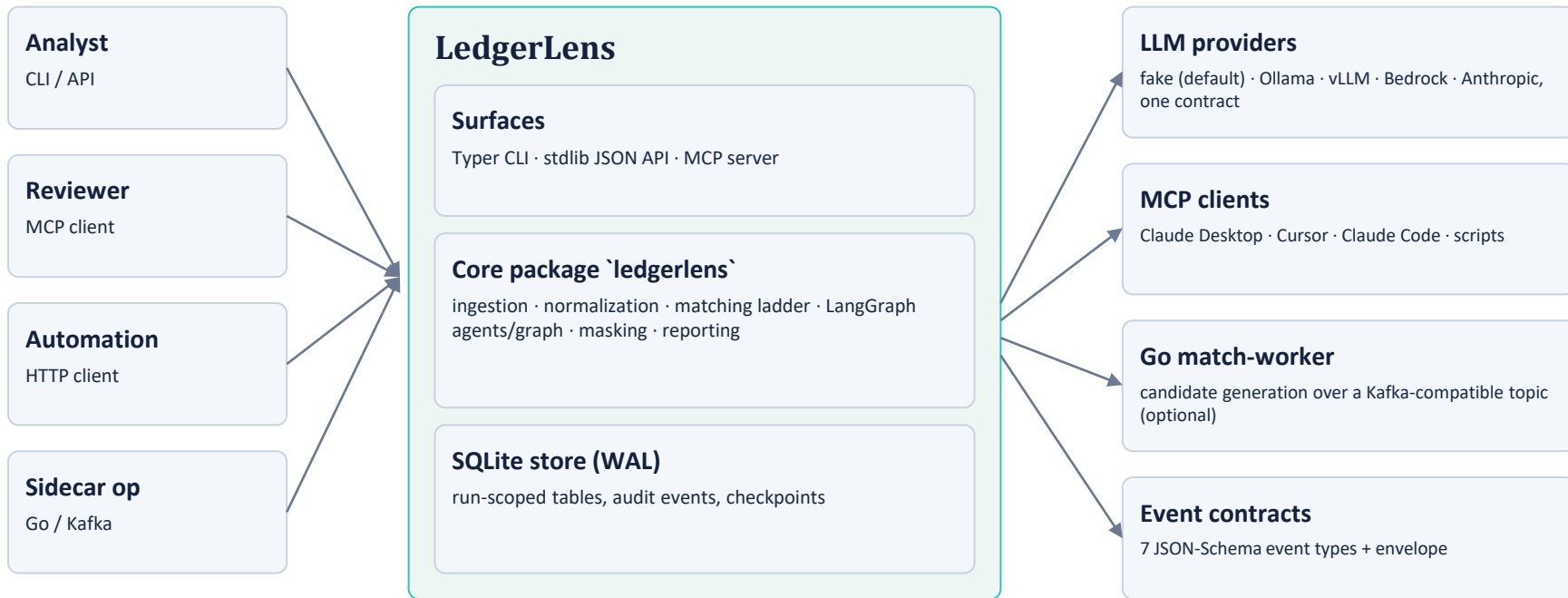
Surfaces

Typewriter CLI · stdlib JSON API · MCP server (`ledgerlens-review`, 7 tools). All three call the same service functions over the same SQLite store; none re-implements the graph.

One perimeter, two consumers

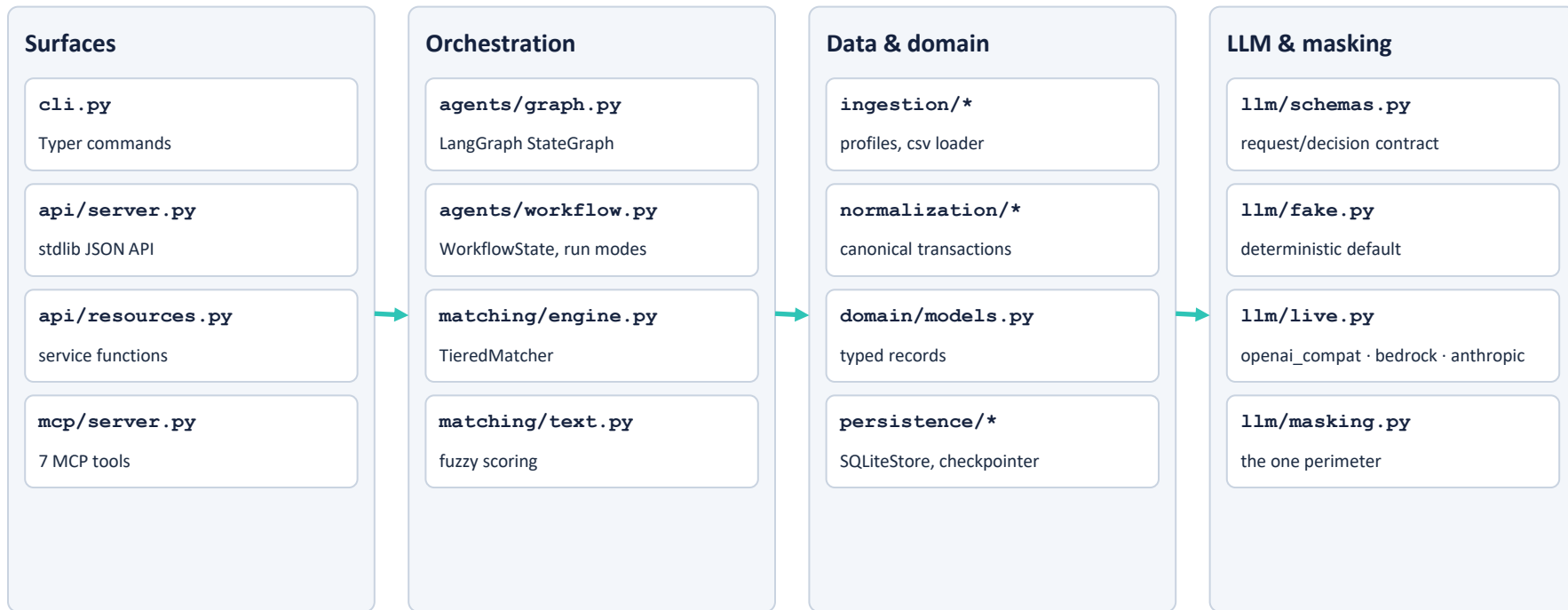
`ledgerlens/llm/masking.py` masks every candidate pair the same way for the model and for the MCP reviewer's tools: hashed tokens for reference and counterparty, digit-run redaction, a version tag.

System context



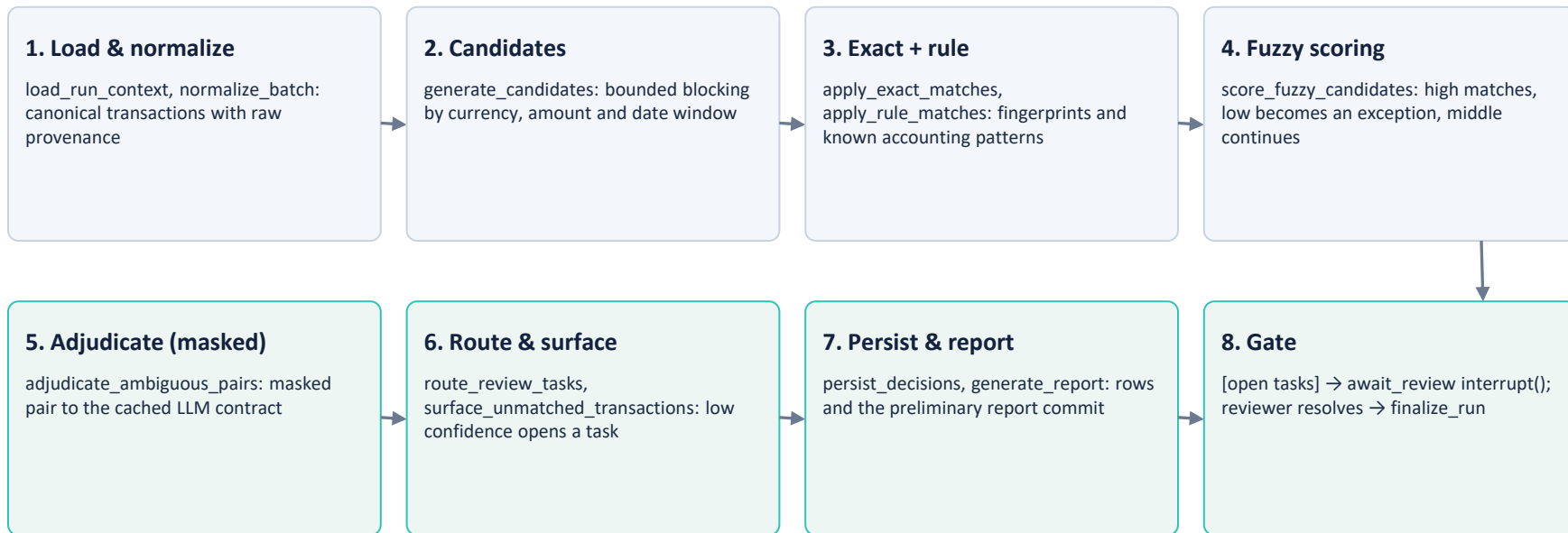
The fake adjudicator is the default and drives every test, the golden replay and CI; live providers are switched on by `--llm {ollama|vllm|bedrock|anthropic}` and a config file naming an environment variable, never a key.

Component architecture



Dependency direction is left to right. The matching engine is stdlib-only; LLM and masking are the only modules a candidate pair's raw fields ever reach on the way out.

The reconciliation graph



Thirteen nodes, one fixed order, no loops and no free-form tool calls. On the acme replay: 2 exact, 1 rule, 1 llm→needs_review, 1 human resolution; 4 of 12 transactions stay unmatched exceptions.

Deterministic first, masked LLM last: the perimeter

What crosses the perimeter (masked pair)

```
{ "id": "txn_23e4...", "date": "2026-05-04",  
  "amount": "-25.00", "currency": "USD",  
  "source_system": "bank",  
  "description": "bank service fee may",  
  "reference_token": "",  
  "counterparty_token": "aabfc736adc8",  
  "masking_version": "ledgerlens.masking.v1" }
```

What the module guarantees

- id, date, amount, currency, source_system stay verbatim
- reference and counterparty become stable hashed tokens (equal tokens still mean equal values)
- description: digit runs of five or more redacted, capped at 80 chars
- the fake reads computed features only, so masking moved none of the golden numbers

exact · rule · fuzzy

resolved without the model — 3 of 4 decisions on the acme replay

llm: needs_review

masked pair, cached by pair, opens a review task

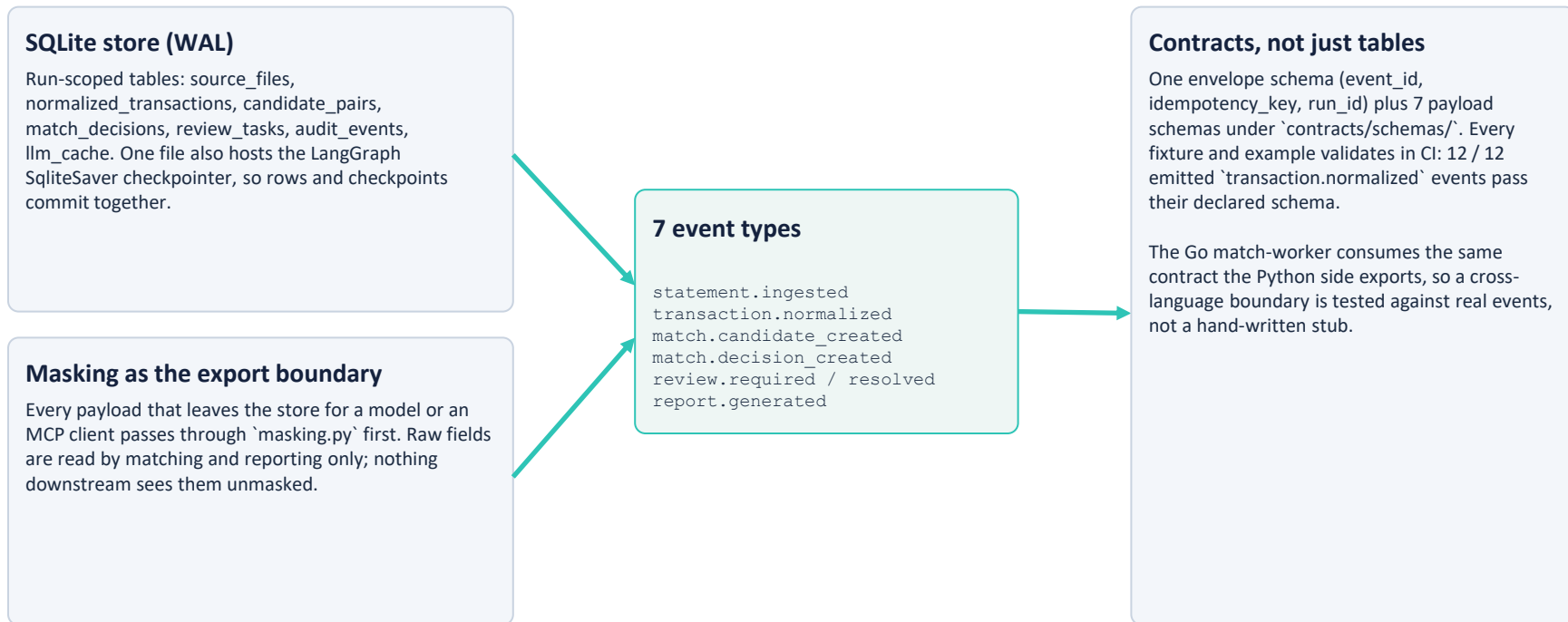
human: match / no_match

final decision; audited; the run cannot complete while it is open

Why this matters

Only the ambiguous middle band ever reaches the model, and the model never sees the pair twice: `CachedLLMAdjudicator` keys on the pair and the model family, so fake and live decisions never collide.

Data architecture: SQLite of record, events at the edge



Classification: masked/tokenised fields only ever leave the store for a model or a review tool; raw customer data stays inside the SQLite boundary and the process that reads it.

LLM providers: one contract, five backends

--llm	Reaches	Environment
fake	nothing (deterministic)	default; drives tests, golden replay, CI
ollama	openai_compat → localhost:11434/v1	no key; stdlib urllib
vllm	openai_compat → localhost:8000/v1	VLLM_API_KEY
bedrock	boto3 converse, us-east-1	AWS credential chain; [bedrock] extra
anthropic	Anthropic Messages API	ANTHROPIC_API_KEY; [anthropic] extra

One `LLMClient` protocol

`build_prompt` states the task, the three allowed decisions and the exact JSON shape; `parse_decision` tolerates fences and prose. A live client retries once with the parse error appended, then raises `LLMError`.

Config, never secrets

`configs/llm/<name>.json` holds the backend, model, base URL or region and the **name** of the key's env var — never a value. A missing variable or package raises `LLMError` naming what to set.

Cache-key separation

`model_family` (`<backend>:<model>`) enters the cache key, so a live decision and the fake's decision for the same pair never collide.

None of the three live backends has been run by the harness; their accuracy, latency and cost are pending — the results card says so rather than estimating.

Design choices and why

Decision	Alternatives considered	Why this one
LangGraph StateGraph + SqliteSaver (D-1)	Hand-rolled node loop (0.1's actual approach)	A checkpointer makes the pause resumable from any process, not just a status flag
Checkpoints hold ids and counters only (D-2)	Checkpoint the full WorkflowState	The store is the record; the checkpoint is the cursor — small, cheap, replayable
The gate finalises, it never re-adjudicates (D-3)	Re-run adjudication after review	Reviewer authority is final and auditable; the model is not consulted twice
Resume validated in the service, not the node (D-4)	Validate inside `await_review`	LangGraph persists the resume value before the node runs; a raising node would wedge the thread
One masking module for the model and MCP (D-5)	Separate redaction per consumer	One version tag, one perimeter; the fake is provably unaffected so golden numbers hold
`openai_compat` over stdlib `urllib` (D-6)	Vendor SDKs for Ollama and vLLM	Ollama and vLLM need nothing installed; Bedrock/Anthropic stay optional lazy imports
SQLite now, Postgres-shaped schema for later	Postgres from day one	Portable local demo; the run-scoped table design maps directly to a production database

Major features

Config-driven source onboarding

A new client source is a mapping profile under ``configs/clients/`` plus one focused test — no matching-code change.

The tiered matching ladder

Exact fingerprint → deterministic rules (date lag, reference variants) → fuzzy scoring with an explicit middle band.

Masked, cached LLM adjudication

Only the ambiguous band reaches the model; decisions are cached per pair so a re-run never asks twice.

The LangGraph review gate

A real ``interrupt()`` at ``await_review``; ``complete_review`` resumes the graph from any process once every task is resolved.

The MCP review server

``ledgerlens-review``: 7 tools over stdio for Claude Desktop, Cursor or a script — masked pairs only.

Atomic runs with rollback

A run that fails before the interrupt rolls back completely, checkpoints included.

Go match-worker + event contracts

7 JSON-Schema event types; the worker is validated by replaying real Python-exported events.

Source diagnostics

Missing references, missing external ids, duplicate references — surfaced before matching starts.

The offline gate

``make check``: 105 unit/e2e tests, the golden replay, and a results-card drift guard, no network, no key.

The reconciliation report — and the gate pause

```
ledgerlens demo — reconciliation • review gate pause

# LedgerLens Reconciliation Report

Run: 'run_c6d6c0917adb'

## Executive Summary

- Source files: 2
- Normalized transactions: 12
- Candidate pairs: 4
- Match decisions: 4
- Open review tasks: 1
- Unmatched transactions: 4
- LLM calls made: 1
- LLM cache hits: 0

## Decisions By Tier

- exact:match: 2
- llm:needs_review: 1
- rule:match: 1

## Source Diagnostics

### bank

- Records: 6
- Missing references: 1
- Missing external IDs: 0
- Duplicate references: none

### ledger

- Records: 6
- Missing references: 1
- Missing external IDs: 0
- Duplicate references: none

## Unmatched Transactions

- Left source: 2
- Right source: 2

## Demo Notes

- Deterministic tiers handle clear work before the fake LLM adjudicator is consulted.
- Every decision is persisted with evidence and an audit event.
- Review tasks preserve human authority for low-confidence or duplicate-pressure cases.
- The fake LLM contract keeps tests offline while matching the live integration shape.

## Audit

- batch.normalized: 1
- exceptions.unmatched_detected: 1
- match.candidate.created: 1
- match.decision.created: 4
- report.generated: 1
- review.required: 1
- run.loaded: 1
- run.persisted: 1
Run ID: run_c6d6c0917adb
Status: awaiting_review
```

What you are looking at

`ledgerlens.cli demo` ingests two acme sources, reconciles across every tier and prints the preliminary report: 2 exact, 1 rule, 1 llm review task, 4 unmatched. It ends `awaiting\_review` — the run is paused, not finished.

The tier breakdown is the product

The report states what the model was asked (1 call, 0 cache hits) and what it was not asked — the other 3 decisions never reached it. This is the number that justifies the architecture to an operations lead.

The review gate: refuse, resolve, resume, finalize

Refused, then resolved

`review-complete` refuses while a task is open, with the exact reason. `review-resolve` records the analyst's decision and notes against the task.

Resumed from a fresh process

The second `review-complete` call opens a new workflow instance on the same database file and resumes the interrupted graph: `human:no\_match` appears under Decisions By Tier, `run.finalized` under Audit, status `completed`.

```

===== review gate refuse early, resolve, resume from a fresh process =====

=== try early complete (expect refusal) ===
Error: run_run_cddcd9f7adb still has 1 open review task(s); resolve them first

=== resolve the task ===
Resolved review_run_cddcd9f7adb.tier.ref11c48719129a.tier.bdbf80ac2791b23 as no_match
=== review-complete (fresh process resumes the interrupted graph) ===
# LedgerLens Reconciliation Report

Run: 'run_cddcd9f7adb'

## Executive Summary

- Source files: 2
- Normalized transactions: 12
- Candidate pairs: 4
- Match decisions: 2
- Open review tasks: 0
- Unmatched transactions: 4
- LIA calls made: 1
- LIA cache hits: 0

## Decisions By Tier

- exact match: 2
- human no match: 1
- lia:needs_review: 1
- rule match: 1

## Source Diagnostics

### bank

- Records: 6
- Missing references: 1
- Missing external IDs: 0
- Duplicate references: none

### ledger

- Records: 6
- Missing references: 1
- Missing external IDs: 0
- Duplicate references: none

## Unmatched Transactions

- Left source: 2
- Right source: 2

## Data Notes

- Deterministic tiers handle clear work before the fake LIA adjudicator is consulted.
- Every decision is persisted with evidence and an audit event.
- Review tasks preserve human authority for low-confidence or duplicate-pressure cases.
- The fake LIA contract keeps tests offline while matching the live integration shape.

## Audit

- batch.normalized: 1
- decision_created: 1
- exceptions.unmatched_detected: 1
- match.consultation_created: 1
- match.decision_created: 4
- report.generated: 1
- review.required: 1
- review.resolved: 1
- run.finalized: 1
- run.loaded: 1
- run.persisted: 1

Run ID: run_cddcd9f7adb
Status: completed

```

```
graph-history -- 15 LangGraph checkpoints, oldest first

-1 __start__ - 1f1af03a-25e1-68dc-bfff-82f46c9827b8 2026-09-12T123:42:36.993443+00:00
0 __start__ 1f1af03a-25e5-6cf1-8080-df41b894a591 2026-09-12T123:42:36.995188+00:00
1 load_run_context load_run_context 1f1af03a-25ec-6dfl-8001-fc246c748696 2026-09-12T123:42:36.998081+00:00
2 normalize_batch normalize_batch 1f1af03a-25ef-6cf2-8002-993b24c663f7 2026-09-12T123:42:36.999265+00:00
3 generate_candidates generate_candidates 1f1af03a-2605-6761-8003-462974f8db48 2026-09-12T123:42:36.918153+00:00
4 apply_exact_matches apply_exact_matches 1f1af03a-2606-6b6e-8004-d6efc7f7384c 2026-09-12T123:42:36.918667+00:00
5 apply_rule_matches apply_rule_matches 1f1af03a-2608-6e68-8005-9c498c38b064 2026-09-12T123:42:36.920791+00:00
6 score_fuzzy_candidates score_fuzzy_candidates 1f1af03a-2609-6e69-8006-477797a6c3d3 2026-09-12T123:42:36.920791+00:00
7 adjudicate_unambiguous_pairs adjudicate_unambiguous_pairs 1f1af03a-2610-6cf0-8007-d09016f84e84 2026-09-12T123:42:36.920802+00:00
8 review_review_tasks review_review_tasks 1f1af03a-2610-6cf5-8008-c4d6016b212a 2026-09-12T123:42:36.922082+00:00
9 surface_unmatched_transactions surface_unmatched_transactions 1f1af03a-2610-6cf6-8009-0a9d3c368ac 2026-09-12T123:42:36.922082+00:00
10 persist_decisions persist_decisions 1f1af03a-2615-6b7c-800a-26ba170bd015 2026-09-12T123:42:36.920812+00:00
11 generate_report generate_report 1f1af03a-261a-6a09-800b-aabf58846c845 2026-09-12T123:42:36.926823+00:00
12 await_review await_review 1f1af03b-8c4e-6998-800c-4aa72ac343c2 2026-09-12T123:43:01.064280+00:00
13 finalize_run finalize_run 1f1af03b-8c4d-6773-800d-187ad82c803f 2026-09-12T123:43:01.064894+00:00
```

graph-history: 15 checkpoints for the gated demo, oldest first — every step of the run is replayable from the checkpoint.

[illegible]

make check, unpacked: 105 tests, the golden replay's 6 KPIs, the headline self-check, the card-drift guard and the Go worker's own tests.

The MCP review server: masked, zero raw identifiers

```
ledgerlens-review MCP server — masked pair, 0 raw identifiers

$ python -m ledgerlens.cli mcp # ledgerlens-review, 7 tools over stdio

> list_runs {"limit": 5}
{"runs": [{"id": "run_66b5814964e0", "client_id": "acme", "status": "awaiting_review", ...}]}

> get_review_pair {"task_id": "review_run_66b5814964e0_txn_9255d26a6efac4be_txn_23e4d5b336bad2ff"}
{
  "pair": {
    "pair_id": "pair_01d242b2daa9b6f07",
    "left": {
      "id": "txn_23e4d5b336bad2ff", "date": "2026-05-04", "amount": "-25.00",
      "currency": "USD", "source_system": "bank",
      "description": "bank service fee may",
      "reference_token": "", "counterparty_token": "aabfc736adc8",
      "masking_version": "ledgerlens.masking.v1",
      "right": {
        "id": "txn_9255d26a6efac4be", "date": "2026-05-04", "amount": "25.00",
        "currency": "USD", "source_system": "ledger",
        "description": "bank monthly service charge",
        "reference_token": "", "counterparty_token": "",
        "masking_version": "ledgerlens.masking.v1"
      }
    },
    "machine_decision": {"tier": "llm", "decision": "needs_review", "confidence": 0.72,
      "reason_code": "llm_low_confidence"}
  }
}
# no raw counterparty or reference string from data/samples/acme*.csv appears above

> resolve_review_task {"task_id": "review_run_66b5814964e0...", "decision": "no_match",
  "notes": "Different business event.", "reviewer": "mcp-reviewer"}
{"task": {"status": "resolved", "reviewer_decision": "no_match", "assigned_to": "mcp-reviewer"},
  "decision": {"tier": "human", "decision": "no_match", "confidence": 1.0,
    "reason_code": "human_review_resolution"}}

> complete_review {"run_id": "run_66b5814964e0", "reviewer": "mcp-reviewer", "note": "reviewed"}
{"finalized": true, "reviewer": "mcp-reviewer", "run_id": "run_66b5814964e0", "status": "completed"}

> get_report {"run_id": "run_66b5814964e0"}
{"counts": {"audit_events": 14, "candidate_pairs": 4, "match_decisions": 5,
  "normalized_transactions": 12, "review_tasks": 1, "source_files": 2},
  "decisions_by_tier": {"exact:match": 2, "human:no_match": 1, "llm:needs_review": 1, "rule:match": 1}}
```

What a reviewer's tool sees

`get\_review\_pair` returns the masked pair plus the machine's tier, decision, confidence and reason code — never the raw counterparty or reference. A test asserts 10 real strings from the sample CSVs against every tool result.

The same gate, another door

`resolve\_review\_task` and `complete\_review` call the identical service functions the CLI and API use; the run cannot complete with an open task through any of the three surfaces.

Security and compliance controls

Boundary	Risk	Control in the code	Evidence
Masking perimeter	raw counterparty / reference reaching a model or reviewer tool	`llm/masking.py`: hashed tokens, digit-run redaction, version tag; one module for both consumers	`test_masking.py` (12); MCP no-leak test (10 sample strings)
MCP review server	a tool result leaking a raw sample value	every tool masks before returning; errors come back as `{"error": ...}`, never a raised exception	`test_no_tool_result_contains_raw_counterparty_or_reference`
LLM provider config	a key committed to source	`configs/llm/*.json` names an env var only; live backends are lazy imports behind extras	`test_shipped_configs_have_expected_shape_and_no_secrets`
Persistence / rollback	a half-written run after a late failure	`store.transaction()` covers ingestion through `generate_report`; checkpointer shares the connection and commits with it	`test_persistent_workflow_rolls_back_partial_run_on_late_failure`
Event contracts	a malformed or unschematised event reaching the Go worker	envelope + 7 payload JSON Schemas, validated in CI	12 / 12 fixtures pass their declared schema

Residual risks, written down: no authentication in front of the API or MCP server (single-operator prototype); SQLite is not a multi-writer production database (a Postgres path is a design choice, not yet built); live LLM backends are untested against a real model.

Frozen contracts, one gate, a decision log

01 HLD

02 LLD (multi-agent)

03 demo runbook

04 AI brief

05 review-gate design (frozen §3-6)

5

design docs (01-05); doc 05's §3-6 interfaces are frozen contracts for the build

6

numbered decisions (D-1 .. D-6) in the review-gate design, each with its consequence

3

task packs (LL-1, LL-2, LL-3) scoped to one module each, with an acceptance checklist

1

gate command: `make check` runs unit + golden + card-drift; CI runs the same command

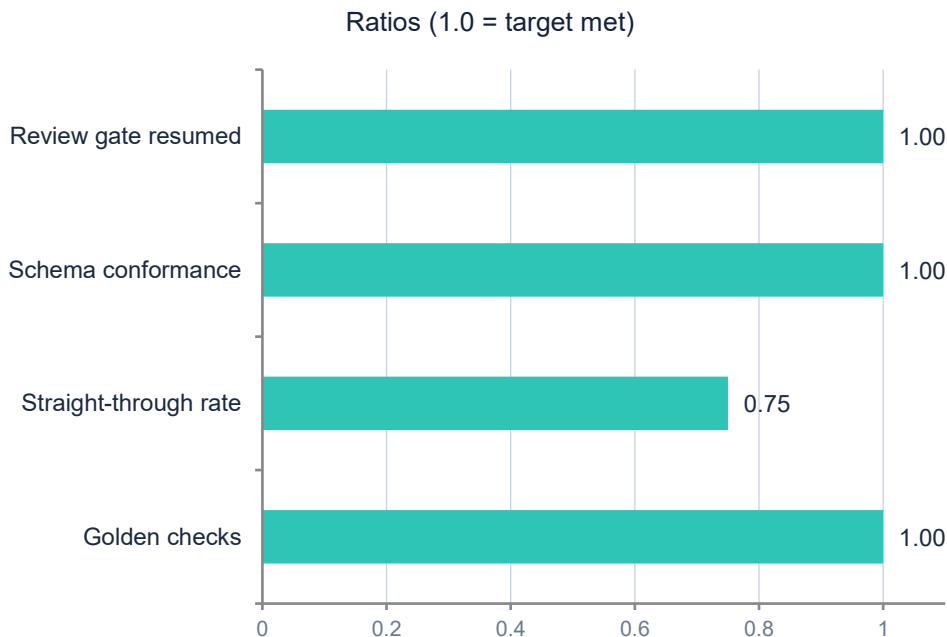
STATE.md tracks the truth

Phase 7, task log LL-0 .. LL-4, each row done/blocked/next; deviations are recorded rather than silently absorbed (0.1 promised LangGraph, 0.2 delivered it — decision D-1).

One command validates everything

``python -m unittest discover -s tests``, ``python -m metrics.golden``, ``python -m unittest tests.golden.test_headline_metrics``, ``python metrics/render.py --check`` — the same four steps `make check` runs, and what CI runs on every push.

Quality metrics from the gate



KPI	Value	Source
Tests	105 passed	unittest discover
Golden checks	6 / 6	vs expected_summary
Match rate	75%	exact + rule + fuzzy
Straight-through	75%	3 of 4 decisions
Review required	25%	1 of 4 decisions
Schema conformance	12 / 12	vs contracts/schemas
Review gate resumed	1 / 1	cross-instance resume
MCP raw-value leaks	0 / 10	sample strings
Go worker packages	2 / 2 ok	go test, GOWORK=off

Live LLM adjudication accuracy is not a KPI here: the harness runs the deterministic fake with no key, so live-model rows in the results card are marked pending, not estimated.

Quirks and known limitations (stated, not hidden)

Live LLM backends are unexercised

Ollama, vLLM, Bedrock and Anthropic ship behind the same contract and are unit-tested against fakes, but no live model has been run by the harness — accuracy, latency and cost are pending.

Only one client pair is golden-replayed

``bank_statement.csv`` / ``ledger_export.csv`` have no mapping profile under ``configs/clients``, so only the acme pair is scored by ``make golden``.

No per-pair precision/recall

``expected_summary.json`` carries aggregate thresholds only; there are no per-pair match labels to score against.

Matching is pairwise only

One-to-many and many-to-many matching are not implemented (tracked in STATE.md's backlog).

Go worker not in the offline gate

``go test`` needs ``GOWORK=off`` locally because ``go.work`` pins Go 1.22 while the worker module requires `>= 1.23`; the worker is validated by ``scripts/verify.py --docker``, not by ``make check``.

No HMAC on review resolutions yet

A resolution is authenticated by the reviewer field only today; signing resolutions is next in STATE.md's backlog.

Masked description keeps its words

Only digit runs of five or more are redacted from the description; short free text like "bank service fee" still reads as English, by design (features, not raw ids, drive the decision).

Streaming profile is opt-in

Redpanda/Kafka validates the boundary, not production throughput; the default demo path is file mode.

Cost, performance and operability

1 call

LLM calls on the acme demo replay; the other 3 of 4 decisions never reach the model

0 keys

needed for tests, the golden replay, CI or the demo; live backends read an env-var name only

15

LangGraph checkpoints for one gated run, each replayable via ``graph-history``

14

audit events recorded for the completed demo run, from ``run.loaded`` to ``run.finalized``

Operability

- ``make check`` — the one offline gate; no network, no key
- ``scripts/verify.py --docker required [--kafka-smoke]`` — Go worker tests, container build, real-event replay, optional Redpanda round trip
- ``docs/03-demo-runbook.md`` — a scripted, minute-by-minute demo with expected output at each step
- Rollback: ``store.transaction()`` covers a run through the interrupt; a late failure leaves nothing partial behind

Observability and audit

- ``audit_events`` table: one typed row per state change (``match.decision_created``, ``review.resolved``, ``run.finalized``, ...)
- ``graph-history`` replays every LangGraph checkpoint for a run, oldest first
- Cache stats (``calls``, ``cache hits``) are printed in every report, so cost is visible per run
- The results card (``metrics/headline.json``) is regenerated by the gate and diff-checked against the committed copy

Roadmap

Now

Record a live run

- Run one real Ollama adjudication and publish it as a recorded figure
- Compare its decision against the fake's on the same masked pair

Next

Matching depth

- One-to-many and many-to-many matching
- HMAC signing on review resolutions
- A second client profile replayed by the golden harness

Then

Production shape

- Postgres migration behind the existing SQLiteStore interface
- Wire the Go worker and a Kafka smoke into CI, not only `scripts/verify.py`
- Authentication in front of the API and MCP server

Later

Scale the review workforce

- Multiple reviewers with role-scoped MCP sessions
- A review-queue dashboard over the same service functions
- SLA and aging metrics on open review tasks

Everything above is additive: the masking perimeter, the review gate and the event contracts do not change.

Appendix A — Decision log (docs/05-langgraph-review-gate-design.md)

ID	Decision	Consequence
D-1	LangGraph + langgraph-checkpoint-sqlite + mcp become core dependencies	The README's dependency-free claim narrows to the matching engine, which stays stdlib
D-2	Checkpoints hold ids and counters only	The store is the record; the checkpoint is the cursor; checkpoints stay small
D-3	The gate finalises the run; it does not re-adjudicate	Reviewer decisions stand; the model is never consulted a second time
D-4	Resume payloads are validated in the service before Command(resume=...)	LangGraph persists the resume value before the node runs; a raising node would wedge the thread
D-5	Masking is one module shared by the LLM request and the MCP server	One version tag, one perimeter; the fake's golden numbers are provably unaffected
D-6	openai_compat uses stdlib urllib	Ollama and vLLM need no extra install; Bedrock/Anthropic stay optional lazy imports

Appendix B — Build statistics and lessons

Item	Value	Source
Tests	105 (unittest, offline)	unittest discover -s tests
Golden checks	6 / 6	metrics/golden.py vs data/golden/expected_summary.json
Design docs	5 (01 HLD, 02 LLD, 03 runbook, 04 AI brief, 05 review-gate)	docs/
Task packs	3 (LL-1 graph-gate, LL-2 live backends, LL-3 MCP server)	docs/tasks/
Numbered decisions	6 (D-1 .. D-6)	docs/05-langgraph-review-gate-design.md
Event contract types	7 + 1 envelope schema	contracts/schemas/
Go packages tested	2 / 2 ok (contracts, worker)	go test ./... , GOWORK=off

Lessons

- A deviation belongs in STATE.md, not silently absorbed: 0.1 promised LangGraph and shipped a hand-rolled loop; 0.2 closed the gap and D-1 records why.
- `go.work` pinning a lower Go version than a module needs is easy to miss locally; the runbook's own Docker command already sets `GOWORK=off` for exactly this reason.
- Validate a resume payload before the graph touches it, not inside the node — LangGraph persists first (D-4), learned the hard way on a sibling project.

Appendix C — Repository map and how to run

```
ledgerlens/      cli, api/, mcp/, agents/ (graph, workflow),
                  ingestion/, normalization/, matching/,
                  llm/ (schemas, fake, live, masking, cache),
                  persistence/, reporting/, domain/
go/match-worker/ contracts, worker, file/kafka transport
contracts/       event envelope + 7 schemas, fixtures
configs/clients/ per-client column-mapping profiles
configs/llm/     backend + model + env-var name, no secrets
data/samples/    acme_bank_statement.csv, acme_ledger_export.csv
data/golden/     expected_summary.json
metrics/         golden.py, headline.json, render.py
docs/           01-05 design docs, tasks/, mcp.md, pitch/
tests/          105 tests: unit, contract, e2e, golden
```

Run it

```
.venv/Scripts/python.exe -m ledgerlens.cli \
  --db .ledgerlens/ledgerlens.db demo

# review the gate
review-list --run-id <run_id>
review-resolve <task_id> --decision no_match \
  --notes "..." --reviewer analyst
review-complete <run_id> --reviewer analyst

# MCP server for a reviewer's tools
python -m ledgerlens.mcp.server

# the offline gate
python -m unittest discover -s tests
python -m metrics.golden
python metrics/render.py --check
```

Questions

LedgerLens · roshanrana/LedgerLens · release 0.2.0 · design docs and the review-gate design in the repository